

Scenario 2: File Upload

Weak Prompt:

- Generate test cases for file upload.

Response for Weak Prompt:

Here is a comprehensive set of test cases for a file upload feature, broken down into functional, user interface, security, and performance categories.

Functional Testing

These test cases ensure the core upload mechanics work according to the business requirements.

- **Valid Upload:** Upload a file with an allowed extension (e.g., .jpg, .pdf) and an acceptable size. Verify the file saves correctly to the destination.
- **Invalid Extension:** Attempt to upload a file with an unsupported extension (e.g., .exe, .bat). Verify the system rejects it and displays an appropriate error message.
- **Max File Size Limit:** Attempt to upload a file that exactly matches the maximum allowed file size. Verify it uploads successfully.
- **Exceeding File Size:** Attempt to upload a file that exceeds the maximum allowed size (e.g., 6 MB on a 5 MB limit). Verify the upload is blocked and an error is shown.
- **Zero-Byte File:** Upload a file that contains no data (0 KB). Verify the system handles it gracefully, typically by rejecting it.
- **Empty Submission:** Click the "Upload" or "Submit" button without selecting any file. Verify the system prompts the user to select a file.
- **Special Characters in Filename:** Upload a file with special characters, spaces, or very long text in the filename (e.g., my f!le @name#.pdf). Verify the system sanitizes the name or processes it without throwing an internal error.
- **Multiple Files (If Applicable):** * Select multiple valid files at once and verify they all upload successfully.
 - Attempt to upload more files than the maximum allowed batch count.

UI / UX Testing

These test cases focus on the user's interaction with the interface.

- **File Explorer:** Verify that clicking the "Browse" or "Choose File" button opens the native operating system file explorer.

- **Drag and Drop:** If supported, drag a file from the desktop into the designated upload zone. Verify the UI updates to show the file is staged.
- **Progress Indicator:** Upload a reasonably large file and verify a progress bar, spinner, or percentage indicator is visible during the process.
- **Success Feedback:** Verify a clear success message (or UI change) appears when the upload completes.
- **Cancellation:** Start an upload and click the "Cancel" button mid-way. Verify the upload stops, the file is not saved, and the UI resets.

Security Testing

These are critical to ensure the file upload feature cannot be exploited by malicious actors.

- **Malware/Virus Upload:** Attempt to upload a known test virus file (like the EICAR test file). Verify the system's antivirus integration catches and rejects it.
- **Front-End Bypass:** Use an intercept proxy (like Burp Suite) or an API tool (like Postman) to bypass the front-end UI validation and send an invalid file type directly to the server. Verify the back-end still blocks the upload.
- **Double Extensions:** Upload a file with a disguised extension, such as `malicious.php.jpg`. Verify the system correctly identifies the true file type or rejects it.
- **Executable Scripts:** Attempt to upload scripts (e.g., `.php`, `.js`, `.py`, `.sh`) to check if the server prevents remote code execution.
- **Path Traversal:** Intercept the upload request and modify the filename to include directory traversal commands (e.g., `../../../../etc/passwd`). Verify the server sanitizes the input and saves the file safely.

Performance & Network Testing

These test cases observe how the feature behaves under stress or poor conditions.

- **Network Interruption:** Start uploading a large file and physically disconnect from the internet. Verify the system shows a timeout or network error gracefully rather than crashing.
- **Slow Network:** Throttle the network speed (e.g., simulate a slow 3G connection using browser dev tools) and verify the upload still completes without timing out prematurely.
- **Concurrent Uploads (Load Testing):** Simulate multiple users uploading files at the exact same time to ensure the server can handle the concurrent processing load without bottlenecking.

ICEPOT Prompt:

- Act as a Senior Manual QA Engineer. Generate manual test cases for a file upload feature where users can upload PDF, JPG, and PNG files up to 5 MB. Include positive, invalid file type, oversized file, empty upload, duplicate upload, error message, and basic security scenarios such as uploading a malicious executable file. For example, verify that a valid PDF file below 5 MB uploads successfully. Present the output in a table with Test Case ID, Scenario, Steps, and Expected Result. Use a professional and technical tone.

Response for ICEPOT Prompt:

Here is a comprehensive manual test suite for the file upload feature based on your specified requirements. The test cases cover functional validation, boundary conditions, negative scenarios, and basic security checks.

File Upload Feature - Test Execution Suite

Pre-conditions: * User is logged into the application (if applicable) and has navigated to the file upload interface.

- Test files of various sizes and formats are prepared in the local test environment.

Test Case ID	Scenario	Steps	Expected Result
TC_FU_001	Positive: Verify successful upload of a valid PDF file under the maximum size limit.	1. Click on the 'Choose File' / 'Browse' button. 2. Select a <code>.pdf</code> file (e.g., 2.5 MB).	The file is uploaded successfully. A success indicator (toast notification or UI state change) is displayed. The file is accessible/visible in the uploaded repository.

		3. Click the 'Upload' or 'Submit' button.	
TC_FU_002	Positive: Verify successful upload of a valid JPG file.	1. Click on 'Choose File'. 2. Select a .jpg or .jpeg file (e.g., 1.2 MB). 3. Click 'Upload'.	The file is uploaded successfully, and a success message is displayed.
TC_FU_003	Positive / Boundary: Verify successful upload of a file that is exactly 5.0 MB (Upper Limit).	1. Click on 'Choose File'. 2. Select a .png file sized exactly at 5,242,880 bytes (5 MB).	The system accepts the file, processes the upload successfully, and displays a success message.

		3. Click 'Upload'.	
TC_FU_004	Negative: Verify validation when attempting to upload an unsupported file type.	1. Click on 'Choose File'. 2. Select an unsupported file type (e.g., <code>document.docx</code> or <code>data.csv</code>). 3. Click 'Upload'.	The upload request is blocked. An explicit error message is displayed (e.g., "Invalid file type. Only PDF, JPG, and PNG are allowed.").
TC_FU_005	Negative / Boundary: Verify validation when uploading a file exceeding the 5 MB limit.	1. Click on 'Choose File'. 2. Select a valid <code>.pdf</code> or <code>.png</code> file sized at 5.1 MB or greater.	The upload request is blocked. An error message is displayed indicating the size limit violation (e.g., "File exceeds the maximum allowed size of 5 MB.").

		3. Click 'Upload'.	
TC_FU_006	Negative: Verify system behavior when attempting an empty form submission.	1. Ensure the file input field is empty (no file selected). 2. Click the 'Upload' button.	The system prevents form submission. A validation error is displayed prompting the user to select a file (e.g., "Please select a file to upload.").
TC_FU_007	Edge Case: Verify system behavior when uploading a duplicate file.	1. Upload a file named <code>report.pdf</code> successfully. 2. Attempt to upload the exact same file (<code>report.pdf</code>) again.	The system handles the collision gracefully. Expected behavior (based on business logic): It either renames the file (e.g., <code>report(1).pdf</code>), overwrites it with user consent, or rejects it with a "File already exists" error.

TC_FU_008	Security: Verify system defense against malicious executables disguised as valid types.	1. Take an executable payload (e.g., <code>script.exe</code> or <code>shell.php</code>). 2. Rename the file extension to bypass client-side validation (e.g., <code>script.exe.pdf</code> or <code>script.pdf</code>). 3. Attempt to upload the file.	The backend MIME-type and content inspection detects the invalid payload. The upload is strictly blocked, the file is discarded, and a security error is logged.
TC_FU_009	Negative: Verify handling of a zero-byte (empty) file.	1. Create an empty <code>.txt</code> file and rename it to <code>empty.pdf</code> (0 KB). 2. Select the file and click 'Upload'.	The upload is blocked. An error message is displayed indicating that the file is empty or corrupted.

QA Notes for Execution:

- Ensure network throttling (e.g., 3G simulation via Chrome DevTools) is utilized on TC_FU_001 to verify UI state (loading spinners/progress bars) during the upload phase.
- For TC_FU_008, verify that the backend API does not rely solely on the `Content-Type` header from the request, as this can be easily manipulated via proxy tools like Burp Suite. True magic byte/file header validation is required.